Centralized Traffic Engineering in Data Center Networks

SmartTE

BACSE203 Computer Networks, Fall 2026
Topic 9

Aayush Santosh Singh	25BCE2341
Ashwin Krishna G	25BDS0038

Contents

1 Part A: Review Paper **2**
1.1 Abstract . . . 2
1.2 Keywords . . . 2
1.3 Introduction and motivation . . . 2
1.4 Distributed routing and weight-based engineering . . . 2
1.5 Topology diversity and data-center multipath . . . 3
1.6 Centralized traffic engineering in WANs . . . 3
1.7 Controller scalability, monitoring, and consistency . . . 4
1.8 Route stability and feedback . . . 4
1.9 Evaluation method and emulation limits . . . 4
1.10 Comparative discussion and research gap . . . 5
1.11 Conclusion and future scope . . . 5

2 Part B: Simulation Case Study **6**
2.1 Topology and controller architecture . . . 6
2.2 Policies and measurements . . . 6
2.3 Results . . . 7
2.4 E4 failure outcomes . . . 10
2.5 Screenshot evidence . . . 14

3 Part C: Alignment and Limitations **14**

4 Team Contributions **15**

1 Part A: Review Paper

1.1 Abstract

Traffic engineering (TE) assigns traffic to network resources to meet performance objectives under changing demand. This review compares distributed shortest-path routing and weight optimization with centralized WAN and data-center approaches. It focuses on what each controller observes, what it changes, and how its evaluation treats operational cost and route stability. These distinctions motivate a bounded SmartTE experiment: whether a periodic port-counter controller with a switching margin and minimum dwell changes path stability and application outcomes in a small emulated topology. The experiment is not a proposal for a new class of TE algorithm and its results are not extrapolated to production networks.

1.2 Keywords

Traffic engineering; software-defined networking; OpenFlow; multipath routing; route stability; Mininet.

1.3 Introduction and motivation

Traffic engineering steers traffic to use available capacity while satisfying service or operational goals. The problem is not simply to find a route: a network may have several feasible paths, but their usefulness depends on current load, traffic demand, measurement freshness, and the cost of changing forwarding state. A design that reacts slowly may leave a bottleneck in place; one that reacts to every noisy measurement may add control traffic or repeatedly move flows.

Data-center fabrics make this tension concrete. Multi-rooted topologies provide multiple paths between hosts, but a path-selection policy determines whether that capacity is used effectively [1, 2]. Software-defined networking (SDN) separates control and forwarding responsibilities, while OpenFlow defines a controller-to-switch interface for flow-table operations [3–5]. Central visibility can coordinate decisions, but it is not free: telemetry, controller processing, rule installation, and transition behavior all matter. The research question for SmartTE is correspondingly narrow: in one two-path emulation, does adding hysteresis and a dwell timer to a counter-based policy reduce successful route changes while retaining useful throughput and delay? The case study measures that implementation under a fixed workload; it does not establish a general result for SDN or TE.

1.4 Distributed routing and weight-based engineering

OSPF is a link-state protocol: routers flood topology information and independently calculate shortest paths using configured link weights [6]. This distributes route computation and failure response, but an operator still has to choose weights that produce desirable network-wide traffic patterns. Fortz and Thorup formulate weight selection as an optimization problem against a demand matrix, illustrating that the configuration problem can be global even when packet forwarding remains distributed [7]. Their later discussion of traditional IP routing describes the practical role of weight-based TE and its constraints [8]. Thus “centralized optimization” and “centralized per-flow forwarding” are different control arrangements: an external optimizer may update costs while each router continues to use its IGP.

Equal-cost multipath (ECMP) uses multiple next hops where shortest-path costs tie. RFC 2992 analyzes hash-threshold selection and the disruption of existing flow assignments when the next-hop set changes [9]. Hashing scales without a controller decision for each flow, but flow-size imbalance and collisions can make utilization uneven. Directly moving selected flows in response to congestion provides more control, at the cost of requiring adequate measurements and update

logic. ECMP's disruption analysis also highlights that a change in route assignments is itself an outcome to measure, not merely an implementation detail.

1.5 Topology diversity and data-center multipath

Fat-tree and related multi-rooted designs use commodity switches to create path diversity between hosts [1]. VL2 develops a scalable data-center architecture and uses Valiant Load Balancing to spread traffic across paths [2]. These works establish architectural opportunities rather than a guarantee that arbitrary traffic will distribute evenly. The assignment rule, traffic matrix, flow sizes, and available path information determine whether nominal capacity can be exploited.

Hedera addresses collisions caused by static flow hashing in multi-rooted data-center networks. It detects large flows and centrally schedules their placement across the fabric; its evaluation includes a commodity-switch implementation and a much larger simulated topology than SmartTE [10]. This approach uses flow-level information and global placement to avoid overlap, while a simple port-counter controller observes aggregate egress activity and may not know which individual flow caused it. The two systems therefore optimize with different information and at different granularities. Hedera's reported simulation result should not be treated as an expected performance level for a four-switch emulation.

MicroTE studies fine-grained TE using short-term traffic patterns and traffic-matrix information [11]. Prediction or repeated traffic structure can enable decisions before congestion is fully visible in a counter interval. That advantage depends on the stability and availability of the workload signal; it is not interchangeable with reactive telemetry. For SmartTE, periodic port counters are deliberately simpler and easier to reproduce, but they can be stale by up to a polling interval and include the controlled flow's own traffic. Moving that flow changes the measurement being optimized, creating a feedback loop absent from a static demand estimate.

CONGA offers a useful contrast within congestion-aware data-center routing: it distributes load-balancing decisions in the network rather than relying on a single centralized controller to select a path for each flow [12]. The approach shows that congestion feedback can be placed at a different point in the system architecture. SmartTE instead uses OpenFlow port statistics collected by OS-Ken and applies one controller policy to the primary flow. These mechanisms are not interchangeable: their measurement locations, switch support, control scope, and evaluated workloads differ.

1.6 Centralized traffic engineering in WANs

B4 reports Google's software-defined WAN design and production experience. Its control system combines traffic engineering with application priorities, demand information, and rate control at the network edge [13]. SWAN also centrally coordinates inter-datacenter traffic, explicitly managing service rates and network configurations; its evaluation combines a testbed prototype with data-driven simulations and studies how to apply changes without transient congestion [14]. These systems show that centralized TE can coordinate more than route choice: rates, priorities, topology constraints, and update sequencing are part of the policy.

The WAN results are valuable architectural evidence, but their operating assumptions differ from this case study. A provider controls endpoints and traffic classes, has a network-wide demand model, and operates across many links and sites. SmartTE uses one primary TCP flow, one optional UDP competitor, two paths, and periodically sampled counters. It does not allocate WAN service rates, optimize a large demand matrix, or provide a distributed high-availability control plane. Accordingly, B4 and SWAN motivate the value of coordinated state while also showing why results from their production, testbed, and data-driven evaluations cannot be compared numerically with SmartTE.

1.7 Controller scalability, monitoring, and consistency

Centralization creates an information advantage but also a scaling question. DevoFlow examines the control-plane and implementation overheads associated with fine-grained flow management and proposes reducing controller involvement for ordinary flows while retaining visibility into significant traffic [15]. This is a different point in the design space from polling a few switch ports once per second, yet the underlying lesson applies: telemetry frequency, message volume, controller work, and update latency are system costs that should be measured. Increasing the polling rate can make the estimate fresher while increasing requests and sensitivity to short-lived variation.

Consistency during a rule update is another distinct concern. Reitblatt and coauthors formalize per-packet and per-flow consistency properties and mechanisms for changing network configurations without exposing packets to an inconsistent mixture of rules [16]. Those guarantees address how a chosen update is installed. They do not determine how often a TE algorithm chooses a new route. Conversely, a hysteresis margin or dwell timer may reduce decision frequency without proving that a multi-switch update is transiently consistent. SmartTE logs decisions and successful transitions, but it does not claim the formal guarantees of consistent-update mechanisms.

1.8 Route stability and feedback

Route stability can refer to several measurable properties: the number of flows remapped after a topology event, repeated oscillation between paths, or transient inconsistency while rules are changed. ECMP disruption analysis treats path remapping after next-hop changes [9]; hybrid-SDN research studies instability caused by interaction between centralized and distributed control loops [17]. The latter is not SmartTE's architecture, but it reinforces the need to distinguish a policy's decision dynamics from the correctness of its forwarding updates.

A switching margin requires an alternative path to be better by a minimum utilization difference; a dwell timer prevents another switch until enough time has elapsed. Both can suppress reactions to small or short-lived differences, but both can also delay a useful route change. In a controller whose score includes the primary flow, changing paths modifies the scores: utilization may fall on the old route and rise on the new one. This endogenous feedback can make a plain best-path policy alternate even if exogenous demand is steady. Therefore, successful route changes should be interpreted alongside throughput, round-trip time, loss, response time, and the measurement method. Fewer changes alone do not prove a better service outcome.

1.9 Evaluation method and emulation limits

Mininet runs real networking software and applications in a lightweight virtual topology, making it useful for reproducible prototyping [18]. Mininet-HiFi evaluates fidelity techniques for container-based emulation and discusses the limits of treating emulated performance as equivalent to a physical network [19]. SmartTE uses Mininet, Open vSwitch, OpenFlow 1.3, and OS-Ken to test controller behavior in a controlled four-switch setup. This supports repeatable comparisons of the declared topology and workload; it does not recreate hardware forwarding, large flow populations, production WAN delay, or controller scaling at fleet size.

Measurement choices also constrain inference. Port-byte deltas estimate link utilization over an interval; iperf3 receiver data measures application throughput; ping supplies sampled delay and loss. These metrics are related but not interchangeable. TCP adjusts its sending behavior in response to congestion, whereas a configured UDP rate offers a different load pattern. Queue limits and host scheduling affect observed outcomes, and short runs can include startup transients. A defensible report therefore names each signal, its clock and aggregation, the number of trials, missing outcomes, and whether intervals are descriptive or part of a statistical

test. The M5 report uses five trials per configuration and descriptive 95% Student-t intervals; it does not claim a hypothesis test.

1.10 Comparative discussion and research gap

Table 1 summarizes the contrasts relevant to SmartTE. The methods are complementary rather than direct competitors: they differ in what they observe, what they control, and how their evidence is produced.

Table 1: Selected traffic-engineering approaches and their relation to SmartTE.

Approach	Main signal and action	Evaluation focus	Difference from SmartTE
OSPF weight optimization [7, 8]	Optimize IGP link weights from a demand matrix; routers then compute shortest paths	Network-wide load under the supplied demand	Changes costs, not per-flow routes
ECMP [9]	Hash each flow over equal-cost next hops	Scaling and path-remapping disruption	No central congestion feedback per flow
Hedera / MicroTE / CONGA [10–12]	Schedule large flows, use traffic patterns, or distribute congestion feedback	Data-center capacity under different workloads	More flow/demand context or in-network coordination than SmartTE
B4 / SWAN [13, 14]	Coordinate WAN demand, priorities, rates, and paths	Production or data-driven WAN utilization	Rate control and operational scale are outside SmartTE
SmartTE B/C	Periodic port-byte deltas; move one h1–h2 flow	Route changes, TCP, ping, and counters in Mininet	A bounded emulation with aggregate measurements

Across these studies, a common theme is that path diversity becomes useful only when traffic assignment matches demand, while the appropriate assignment depends on the available information and control scope. There is no single shared objective: IGP optimization targets network-wide cost, ECMP spreads flows, Hedera schedules large flows, MicroTE exploits traffic patterns, and WAN systems coordinate rates and priorities. Their differing evaluation methods, including analysis, simulation, emulation, testbed, and production deployment, also limit direct numerical comparison.

SmartTE tests a narrower question than these systems: with a common two-path topology and periodic maximum-utilization score, what changes when a lower-score policy is augmented with hysteresis and minimum dwell? The principal gap is not absence of centralized TE or hysteresis in prior work; it is the local measurement of this particular feedback loop across successful route changes and application outcomes under a declared emulated workload. The M5 campaign provides implementation-specific observations only. Broader traffic matrices, more paths and flows, update-consistency mechanisms, hardware tests, and a pre-specified statistical analysis remain future work.

1.11 Conclusion and future scope

The literature shows that centralized TE must balance broader demand awareness against telemetry cost, stale state, route stability, and update safety. SmartTE tests only a small emulated feedback-control case and makes no production or universal-superiority claim. Its 19 sources include 13 peer-reviewed papers and two original standards (15 eligible sources), two informational RFCs, one workshop paper not counted as peer reviewed, and one non-peer-reviewed editorial.

2 Part B: Simulation Case Study

2.1 Topology and controller architecture

The experiment uses four hosts and four Open vSwitch instances. The primary TCP pair is h1–h2; the optional competing UDP pair is h3–h4. Path A is s1–s2–s4, with two 10 Mbps, 5 ms core links. Path B is s1–s3–s4, with two 10 Mbps, 10 ms core links. Access links are 100 Mbps with 1 ms delay. The main comparison uses a 100-packet core queue verified using `tc`.

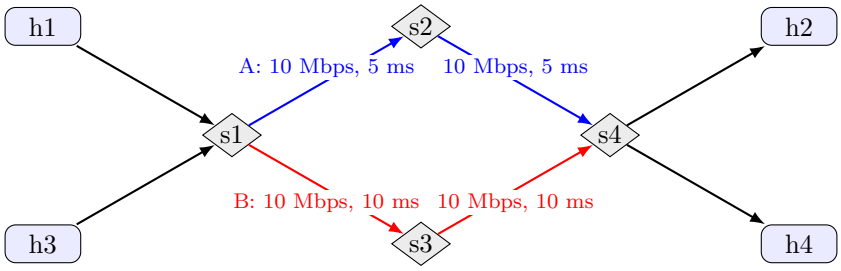


Figure 1: Original vector topology diagram derived from the shared topology specification. Path A is blue and Path B is red.

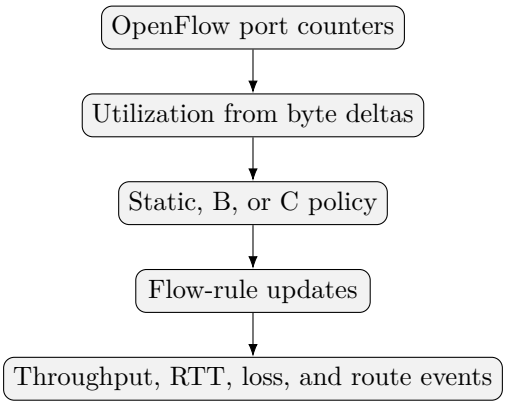


Figure 2: Controller measurement and decision pipeline.

2.2 Policies and measurements

Each path score is the maximum utilization of its two core egress links: s1:3 and s2:2 for A, and s1:4 and s3:2 for B. For each counter interval,

$$U(\%) = \frac{\Delta \text{tx_bytes} \times 8}{\Delta t \times \text{capacity}_{bps}} \times 100.$$

Static retains its configured path. Policy B selects a measurably lower score; Policy C additionally requires a 15 percentage-point improvement and a 5,000 ms minimum dwell in the frozen reference configuration. The M5 poll interval is 1,000 ms.

E1 has no UDP competitor. E2 adds sustained 5 or 9 Mbps UDP on Path A. E3 uses three five-second 9 Mbps bursts beginning at 5, 15, and 25 seconds of a 30-second TCP run and declares C sensitivity variants. E4 disables s2–s4 one second into a 15-second run. Receiver-side iperf3 supplies TCP throughput; ping supplies RTT/loss; OpenFlow counters supply utilization.

2.3 Results

The completed campaign m5_campaign_20261011T115819Z contains 110 planned and recorded rows: 105 `ok`, five `failed`, and no aborted/excluded rows. The five failed rows are the expected static E4 outage controls. Processed 95% Student-t intervals are descriptive; no significance test was performed.

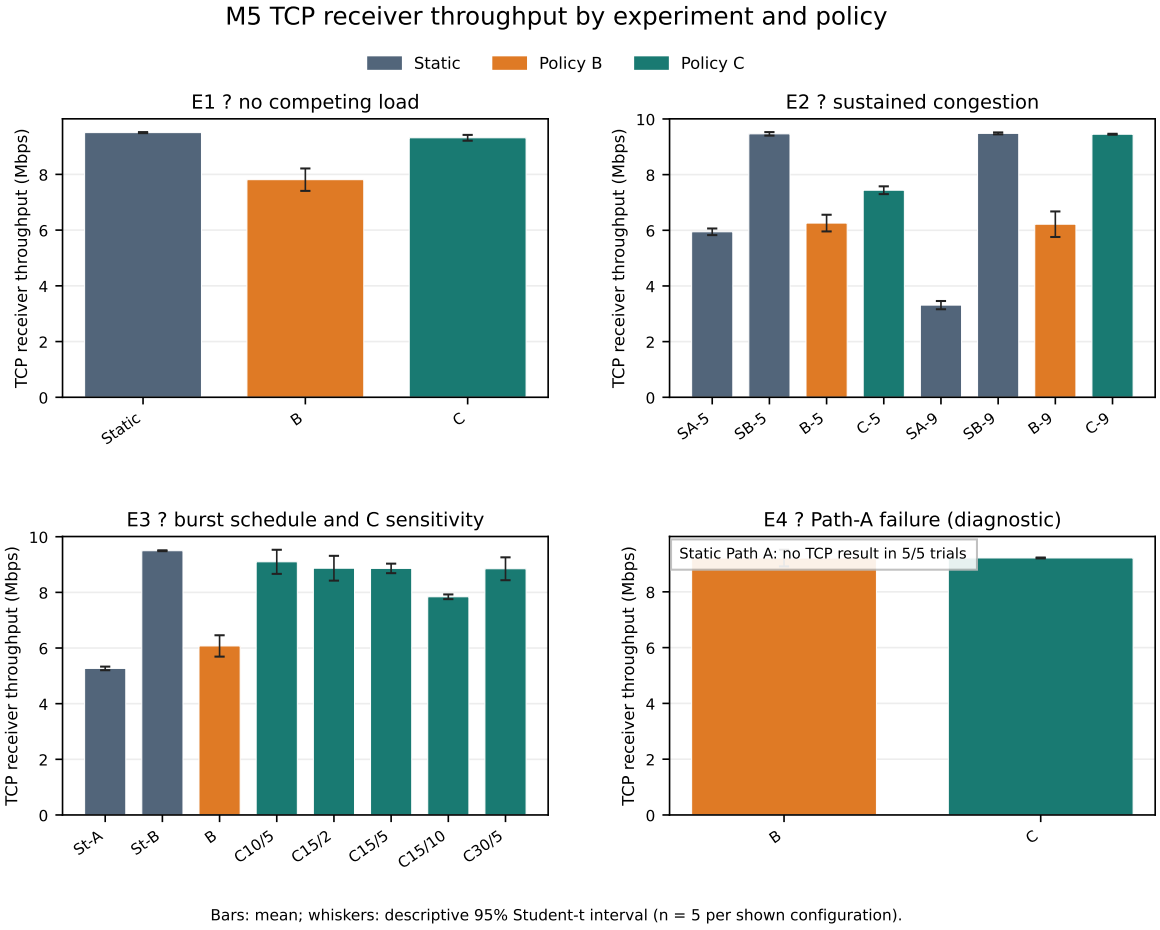


Figure 3: TCP receiver throughput from the M5 processed summary. Bars show means and whiskers descriptive 95% Student-t intervals. E2 labels are policy/load in Mbps (SA/SB denote Static on A/B); E3 C labels are margin points/dwell seconds. E4 Static on Path A has no TCP result in five outage trials.

In E2 with 5 Mbps UDP, Static-on-A averaged 5.946 Mbps TCP receiver throughput, while Static-on-B averaged 9.465 Mbps (five trials each). In E1, B averaged 16 successful route changes per trial and C averaged three. E4 requires caution: B had already changed to Path B before the failure in all trials, while C was on A at failure and changed after it. Raw E4 switch-decision counts include rejected requests; successful route changes are taken from the event audit.

The Static contrast isolates which path carries the competing flow. In E2 at 5 Mbps UDP, Static-on-A averaged 5.946 Mbps TCP, while Static-on-B averaged 9.465 Mbps (n=5 each); at 9 Mbps, the corresponding means were 3.311 and 9.482 Mbps. Values are rounded to three decimals from the M5 `metrics_summary.csv` means (3.3114 and 9.4816 Mbps), grouped by scenario and algorithm.

The source evidence PDF uses its own figure numbering, separate from the numbering generated in this report. Source evidence Figures 1–2 are live terminal captures; Figures 3–

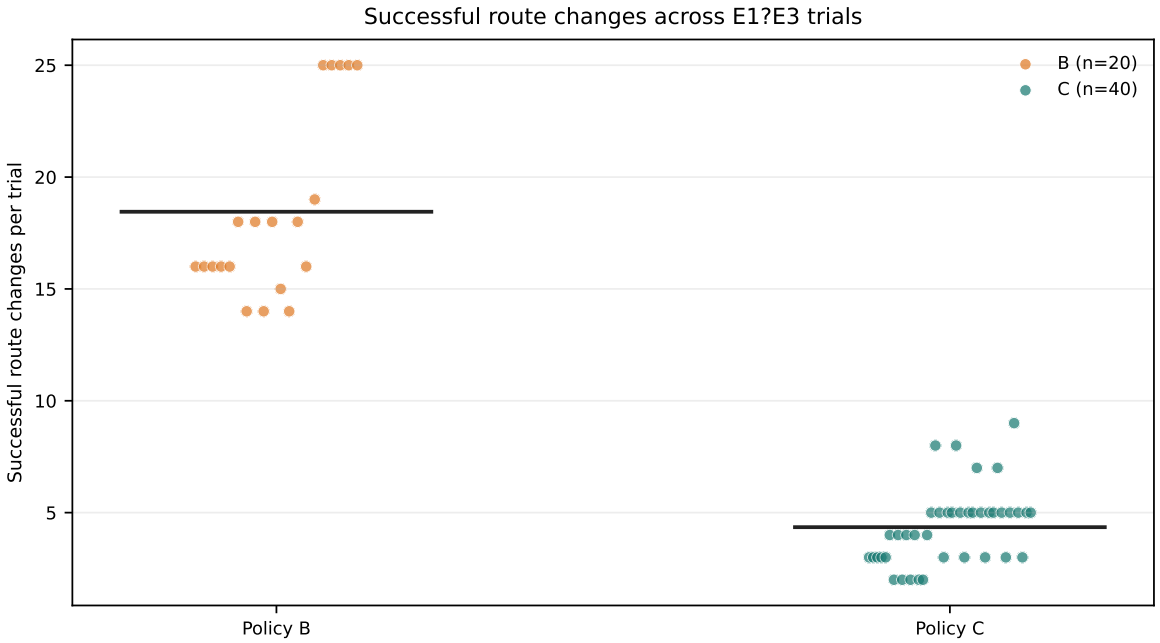


Figure 4: Verified successful route-change comparison for E1–E3. E4 is excluded because its path state at failure differs by policy.

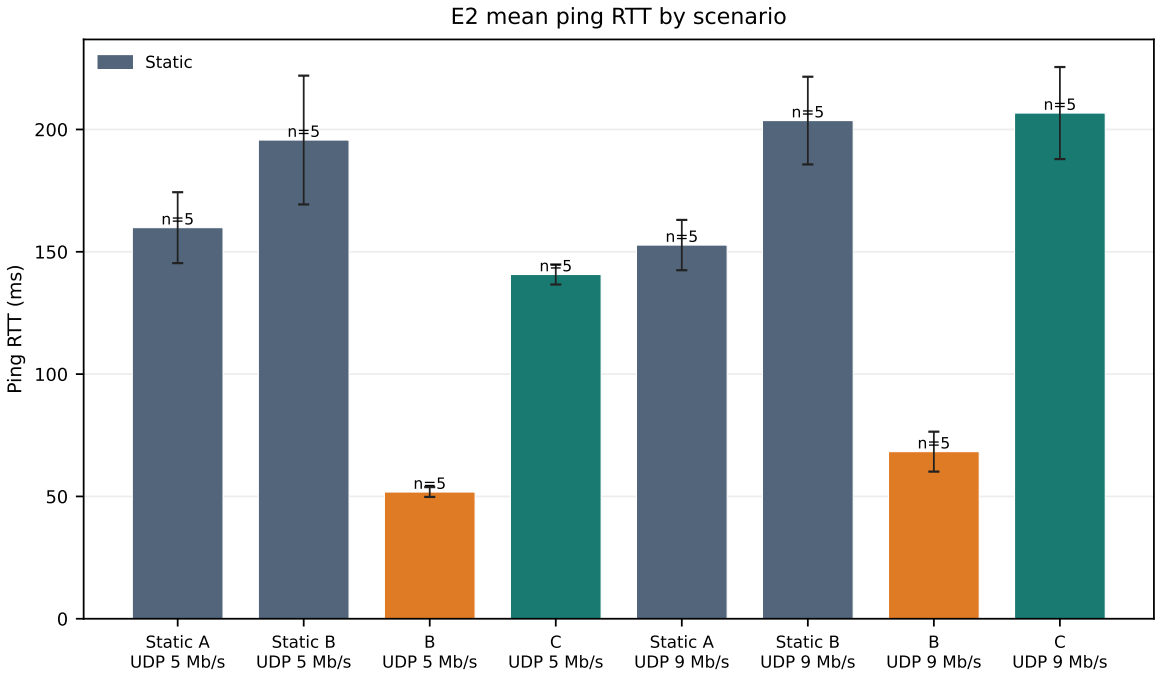


Figure 5: Verified E2 mean ping-RTT figure. Latency and loss vary by scenario; no universal latency benefit is claimed.

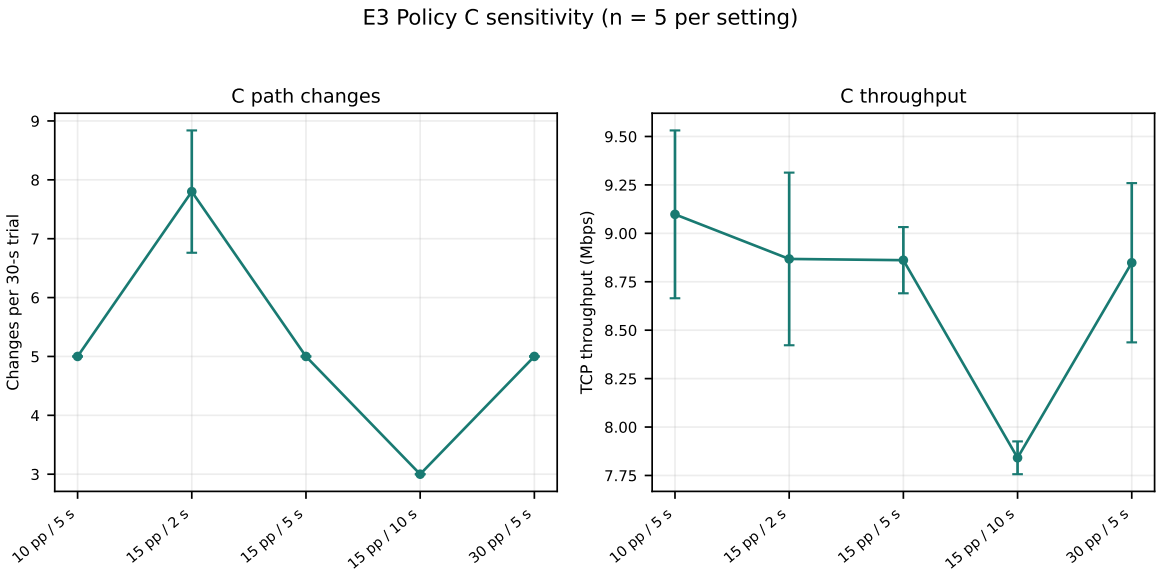


Figure 6: Verified E3 sensitivity figure for Policy C. The declared settings describe a trade-off and do not identify a universally best parameter pair.

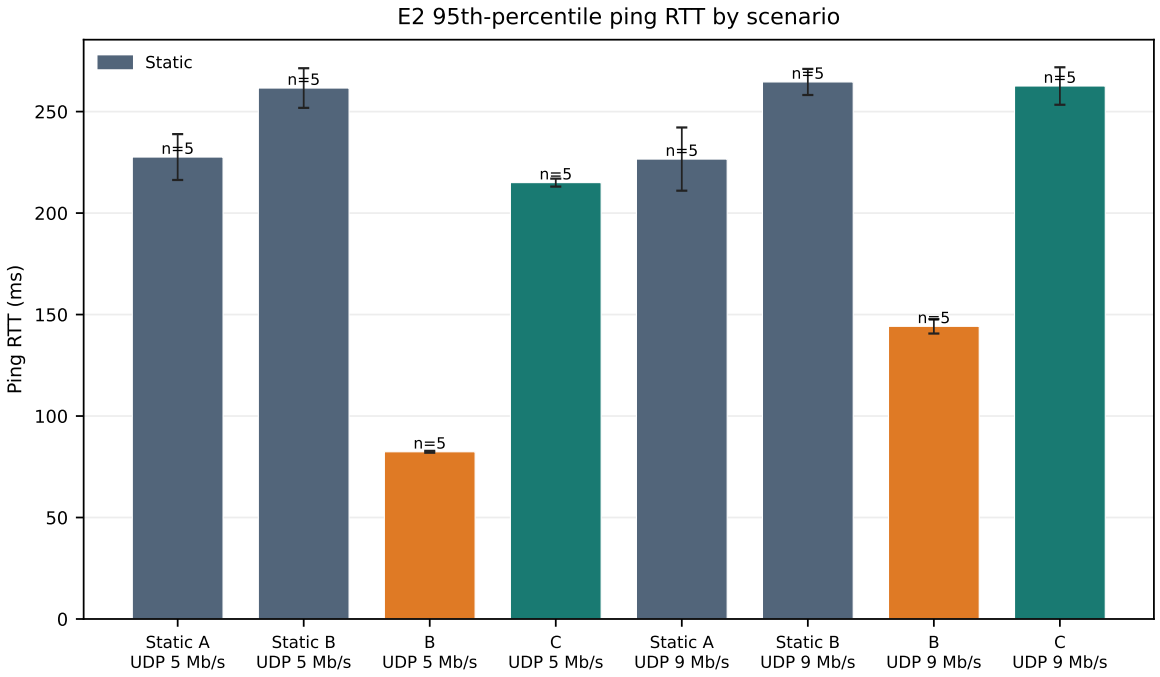


Figure 7: E2 95th-percentile ping RTT from the processed campaign summary.

Table 2: E2 static-path congestion contrast (M5 processed means; n=5).

UDP offered (Mbps)	Primary path	TCP receiver (Mbps)	Mean RTT (ms)
5	A	5.946	159.86
5	B	9.465	195.68
9	A	3.311	152.75
9	B	9.482	203.64

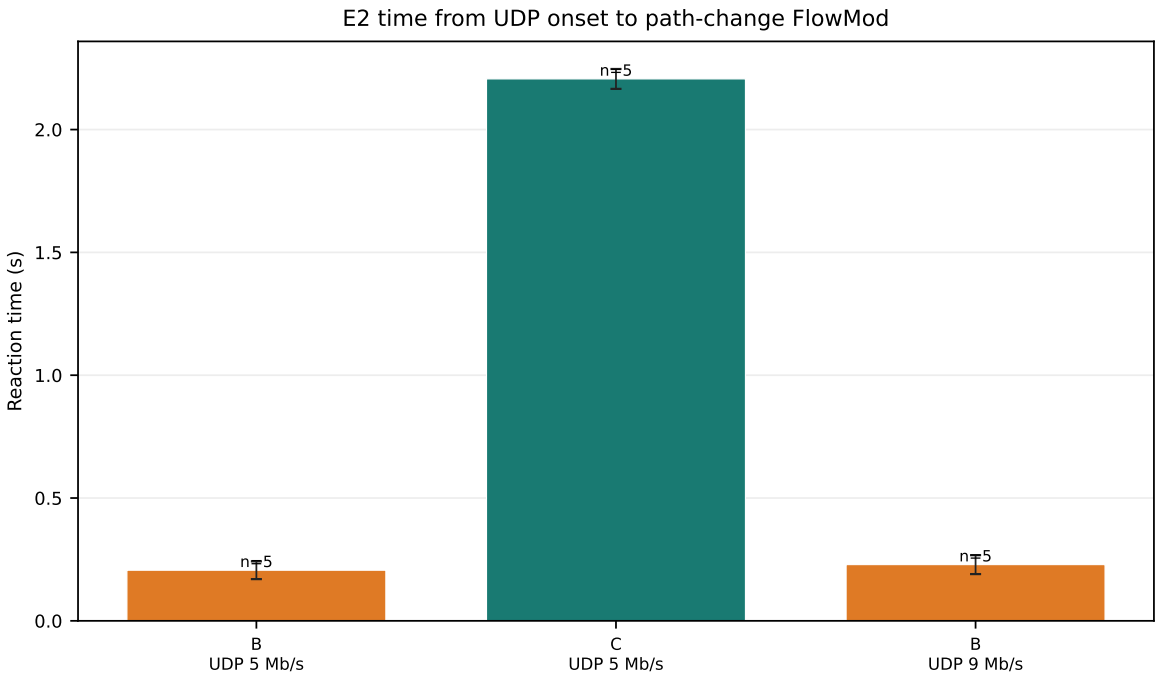


Figure 8: E2 time from UDP onset to the first route-change FlowMod. Empty cells indicate no qualifying event was recorded.

11 are archived evidence, including Figure 9, which is an archived M5 plot despite the PDF's erroneous opening note. Source Appendix Figures A1–A2 show earlier milestone validation, not M5 measurements.

```
andy2341@KUTU:~/smartte$ cd ~/smartte
uname -srm; lsb_release -d
python3 --version; mn --version; ovs-vsctl --version | head -1
~/.local/share/smartte/os-ken-py310/bin/python -c "import sys, importlib.metadata as m; print('OS-Ken', m.version('os-ken'),
 '| Python', sys.version.split()[0])"
Linux 6.6.87.2-microsoft-standard-WSL2 aarch64
No LSB modules are available.
Description:    Ubuntu 24.04.5 LTS
Python 3.12.3
2.3.0
ovs-vsctl (Open vSwitch) 3.3.9
OS-Ken 4.2.2 | Python 3.10.22
```

Figure 9: Live environment capture supplied by the team (source evidence Fig. 1). It records WSL2, Mininet, OVS, Python, and OS-Ken versions.

Queue configuration evidence. The source evidence Fig. 7 is cropped and is not reproduced. The archived E2 static 9-Mbps trial's manifest and full per-interface `tc` output record `queue_limit_verified=true` and the 100-packet netem limit. This confirms the recorded configuration, not transient queue occupancy.

2.4 E4 failure outcomes

Table 3 separates policy outcome measures from controller decision counts. Successful transitions and decision attempts come from the processed event audit; recovery is counter-confirmed time after the link-down event.

The M5 metadata records campaign ID `m5_20261011T115819Z`, commit `58acacd120a1b8b85b81d8a46c1f273479d18942`, and `git_dirty=true`. The dirty flag means that commit alone does not identify the complete source state used during execution; the original metadata and trial evidence are preserved.

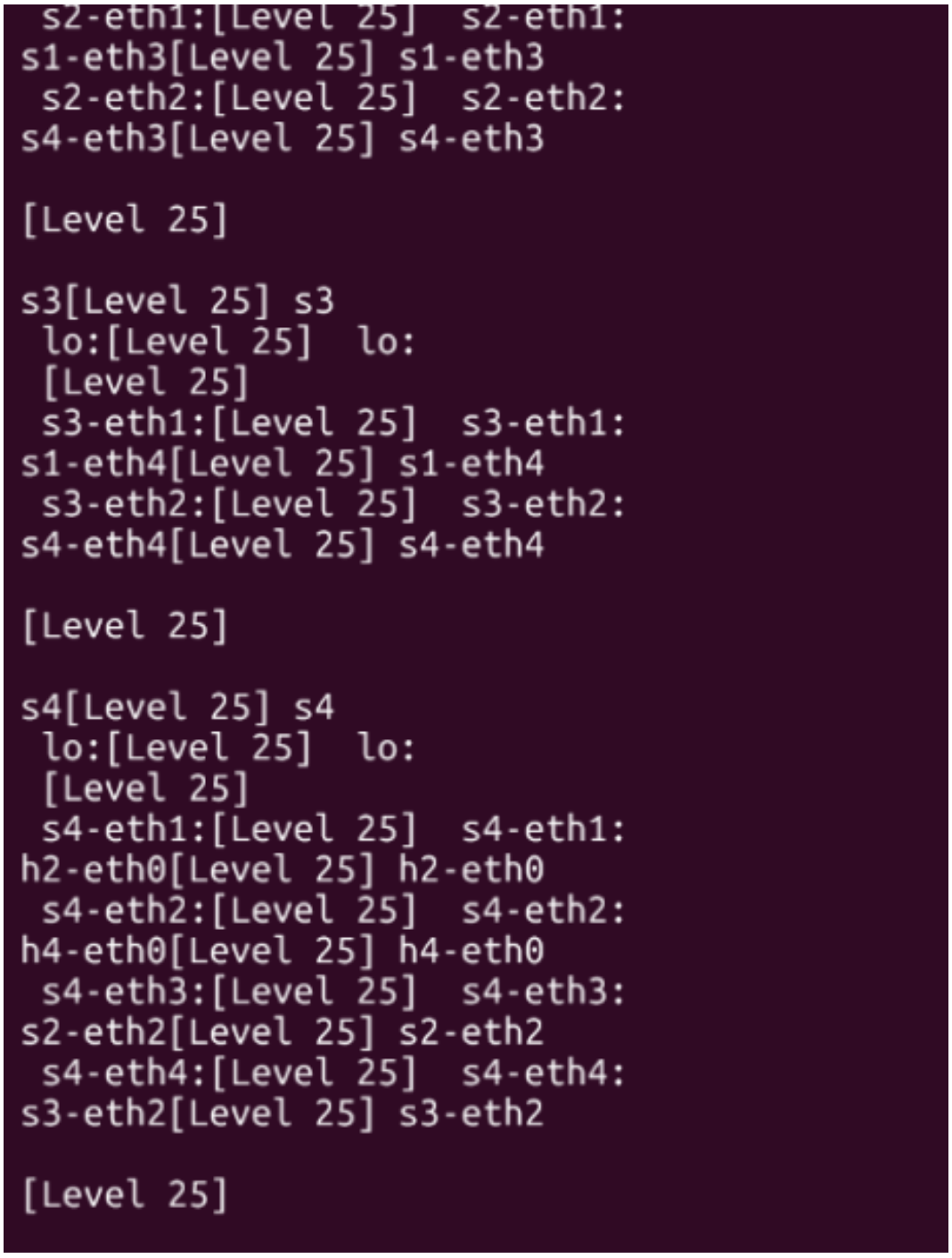


Figure 10: Archived Mininet node and port listing (source evidence Fig. 3). This corroborates the host and switch port mapping used by the controller.

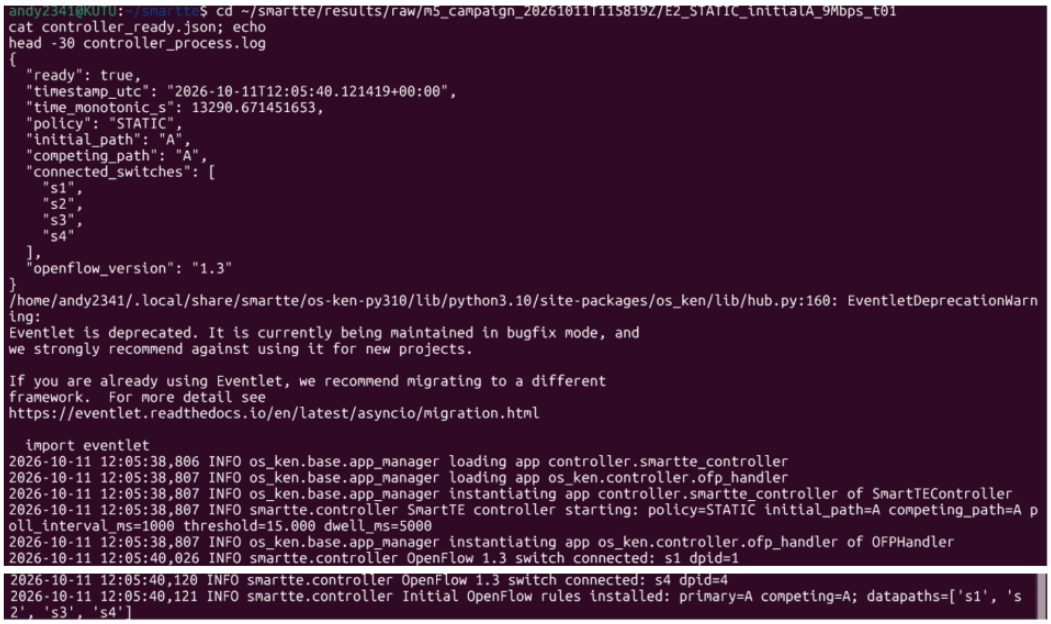


Figure 11: Archived OS-Ken startup log from M5 trial E2_STATIC_initialA_9Mbps_t01 (source evidence Fig. 4); it shows the controller ready state and connected switches.

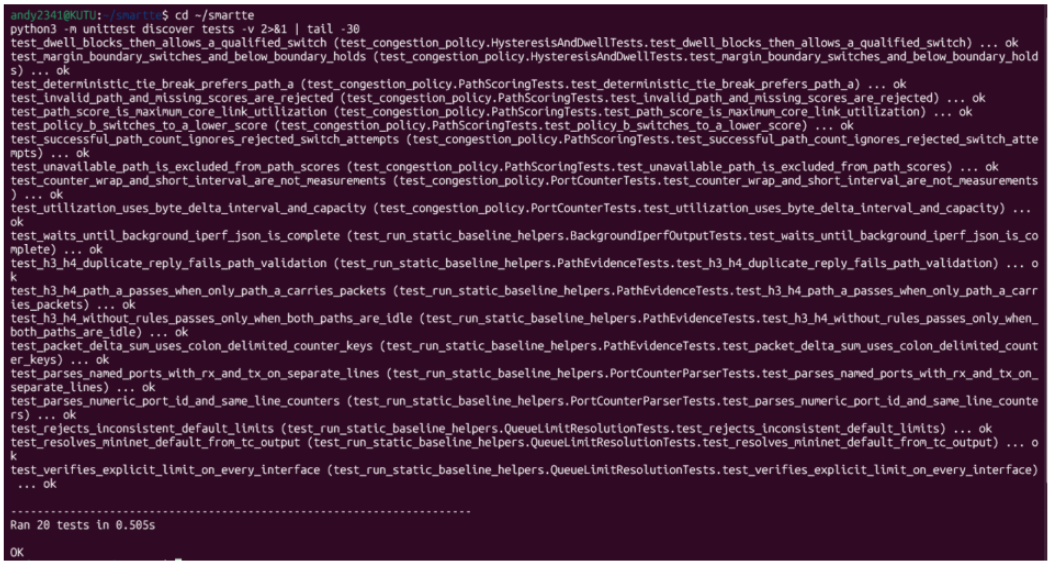
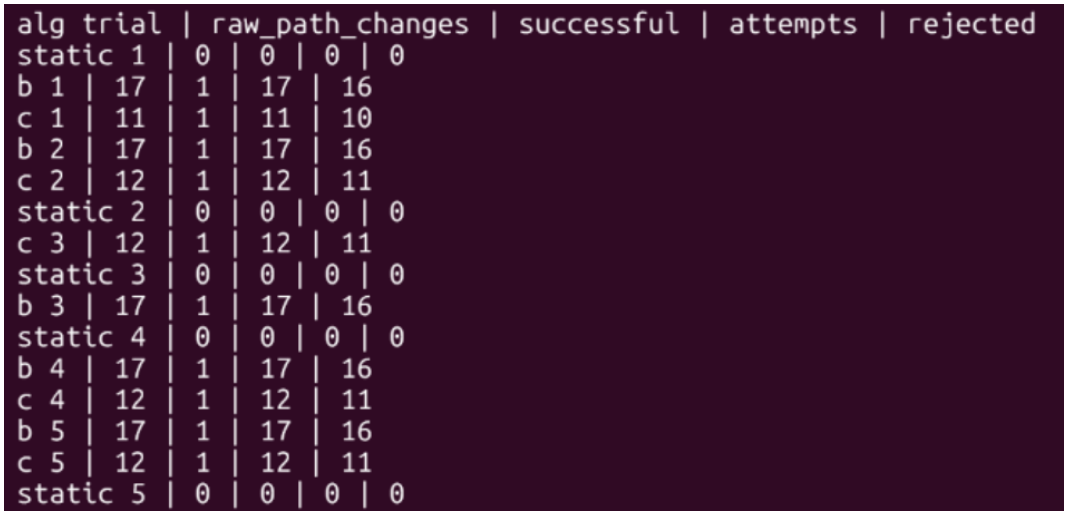


Figure 12: Archived unit-test output showing 20 tests completed successfully (source evidence Fig. 8).



```
alg trial | raw_path_changes | successful | attempts | rejected
static 1 | 0 | 0 | 0 | 0
b 1 | 17 | 1 | 17 | 16
c 1 | 11 | 1 | 11 | 10
b 2 | 17 | 1 | 17 | 16
c 2 | 12 | 1 | 12 | 11
static 2 | 0 | 0 | 0 | 0
c 3 | 12 | 1 | 12 | 11
static 3 | 0 | 0 | 0 | 0
b 3 | 17 | 1 | 17 | 16
static 4 | 0 | 0 | 0 | 0
b 4 | 17 | 1 | 17 | 16
c 4 | 12 | 1 | 12 | 11
b 5 | 17 | 1 | 17 | 16
c 5 | 12 | 1 | 12 | 11
static 5 | 0 | 0 | 0 | 0
```

Figure 13: Archived E4 event audit (source evidence Fig. 11). It separates raw switch decisions from successful transitions and rejected attempts.

Table 3: E4 Path-A link-down outcomes (five trials per policy).

Policy	TCP results	Path B at failure	Successful changes/trial	Attempts per trial	Rejected per trial
Static	0/5	0/5	0	0	0
B	5/5	5/5	1	17	16
C	5/5	0/5	1	11.8	10.8

Policy	Mean counter-confirmed recovery (s)	Mean ping loss (%)
Static	n/a	93.33
B	0.193	2.13
C	1.005	1.87

2.5 Screenshot evidence

The original seven-page evidence PDF is preserved at `evidence/SmartTE_screenshots.pdf`; selected images were rasterized at 2x page resolution into `evidence/extracted/`. Source evidence Figures 2 (links) and 5 (static baseline) are not repeated because the vector topology and processed values communicate those details directly. The archived B-versus-C plot is Report Figure 4. Source evidence Figures 1–2 are live terminal captures, Figures 3–11 are archived evidence, and Appendix Figures A1–A2 show earlier milestone validation. The opening provenance note's classification of source Figure 9 as live is incorrect; its caption identifies it as an archived plot.

3 Part C: Alignment and Limitations

SmartTE agrees with the literature in one limited sense: path diversity can make it possible to avoid a local bottleneck when a controller has usable network-state information [10, 11]. It does not reproduce the scale, workload knowledge, rate control, or operational guarantees of B4, SWAN, or large data-center schedulers [13, 14]. In this emulation, the static E2 references establish that moving the primary flow away from the Path-A competitor can preserve throughput.

The observed policy trade-off also matches the motivation for route-stability controls. B changes paths frequently; the frozen C setting makes fewer successful changes in E1–E3, but it may wait longer before acting. This is a behavior of the measured implementation, not proof that a particular margin/dwell pair is optimal. SmartTE does not provide the formal update-consistency guarantees studied in prior work [16]; it records controller events and differentiates successful transitions from rejected decisions.

The interpretation is constrained by a single-host Mininet/OVS emulation under WSL2, a four-switch topology, two paths with unequal configured delays, one TCP pair, and one optional UDP competitor. Emulated timing, Linux queues, host load, and the recorded HTB warning can affect results. Queue limits were verified, but transient occupancy was not continuously measured. E4 is a diagnostic one-link failure: B had already moved to B before failure in all five trials, while C remained on A and changed after failure in all five. Each had one successful change per trial. Raw decision attempts averaged 17 for B and 11.8 for C; rejected attempts averaged 16 and 10.8, respectively. Static stayed on the failed path in all five trials. E4 therefore cannot rank failover latency fairly. The campaign metadata records commit 58acacd120a1b8b85b81d8a46c1f273479d18942 with `git_dirty=true`, so the exact uncommitted source state is not fully established. The results characterize this code, topology, and workload rather than a production traffic-engineering system.

4 Team Contributions

The team confirmed the following division of responsibilities. Each description identifies a member's assigned domain while recognizing the shared work required to complete and validate the project.

Aayush Santosh Singh (25BCE2341): Systems implementation; controller and routing-policy development; OpenFlow integration; topology setup; experiment execution; debugging; and technical validation within his assigned domain.

Ashwin Krishna G (25BDS0038, SPOC): Literature review; comparative analysis; results interpretation and visualization; and report documentation within his assigned domain.

Joint contributions: Overall system design, integration, testing, review of project findings, and preparation for the final presentation and viva.

Repository history, code, raw evidence, and report material provide supporting project context; commit authorship alone is not used as the basis for this confirmed division.

References

[1] M. Al-Fares, A. Loukissas, and A. Vahdat, "A scalable, commodity data center network architecture," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 4, pp. 63–74, 2008.

[2] A. Greenberg, J. R. Hamilton, N. Jain, S. Kandula, C. Kim, P. Lahiri, D. A. Maltz, P. Patel, and S. Sengupta, "VL2: A scalable and flexible data center network," in *Proceedings of ACM SIGCOMM*, 2009, pp. 51–62.

[3] E. Haleplidis, K. Pentikousis, S. Denazis, J. H. Salim, D. Meyer, and O. Koufopavlou, "Software-defined networking (sdn): Layers and architecture terminology," IRTF, Informational RFC 7426, Jan. 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7426

[4] *OpenFlow Switch Specification, Version 1.3.0*, Open Networking Foundation, Jun. 2012. [Online]. Available: https://opennetworking.org/sdn-resources/openflow-switch-specification/

[5] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[6] J. Moy, "Ospf version 2," IETF, Standards Track RFC 2328, Apr. 1998. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2328

[7] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights," in *Proceedings of IEEE INFOCOM*, vol. 2, 2000, pp. 519–528. [Online]. Available: https://www.cs.cornell.edu/courses/cs619/2004fa/documents/ospf_opt.pdf

[8] B. Fortz, J. L. Rexford, and M. Thorup, "Traffic engineering with traditional IP routing protocols," *IEEE Communications Magazine*, vol. 40, no. 10, pp. 118–124, Oct. 2002. [Online]. Available: https://collaborate.princeton.edu/en/publications/traffic-engineering-with-traditional-ip-routing-protocols/

[9] C. Hopps, "Analysis of an equal-cost multi-path algorithm," IETF, Informational RFC 2992, Nov. 2000. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc2992

[10] M. Al-Fares, S. Radhakrishnan, B. Raghavan, N. Huang, and A. Vahdat, "Hedera: Dynamic flow scheduling for data center networks," in *Proceedings of the 7th USENIX Symposium on Networked Systems Design and Implementation*, 2010. [Online]. Available: https://www.usenix.org/conference/nsdi10-0/hedera-dynamic-flow-scheduling-data-center-networks

[11] T. Benson, A. Anand, A. Akella, and M. Zhang, "MicroTE: Fine-grained traffic engineering for data centers," in *Proceedings of the 7th ACM International Conference on Emerging Networking Experiments and Technologies*, 2011. [Online]. Available: https://wisr.cs.wisc.edu/papers/microte.pdf

[12] M. Alizadeh, T. Edsall, S. Dharmapurikar, R. Vaidyanathan, K. Chu, A. Fingerhut, V. T. Lam, F. Matus, R. Pan, N. Yadav, and G. Varghese, "CONGA: Distributed congestion-aware load balancing for datacenters," in *Proceedings of the 2014 ACM Conference on SIGCOMM*, 2014, pp. 503–514. [Online]. Available: https://conferences.sigcomm.org/sigcomm/2014/program.php

[13] S. Jain, A. Kumar, S. Mandal, J. Ong, L. Poutievski, A. Singh, S. Venkata, J. Wanderer, J. Zhou, M. Zhu, J. Zolla, U. H"olzle, S. Stuart, and A. Vahdat, "B4: Experience with a globally-deployed software defined WAN," in *Proceedings of ACM SIGCOMM*, 2013, pp. 3–14. [Online]. Available: https://research.google/pubs/b4-experience-with-a-globally-deployed-software-defined-wan/

[14] C.-Y. Hong, S. Kandula, R. Mahajan, M. Zhang, V. Gill, M. Nanduri, and R. Wattenhofer, "Achieving high utilization with software-driven WAN," *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, pp. 15–26, 2013. [Online]. Available: https://research.google/pubs/achieving-high-utilization-with-software-driven-wan/

[15] A. R. Curtis, J. C. Mogul, J. Tourrilhes, P. Yalagandula, P. Sharma, and S. Banerjee, "DevoFlow: Scaling flow management for high-performance networks," in *Proceedings of ACM SIGCOMM*, 2011, pp. 254–265. [Online]. Available: https://conferences.sigcomm.org/sigcomm/2011/papers/sigcomm/p254.pdf

[16] M. Reitblatt, N. Foster, J. Rexford, C. Schlesinger, and D. Walker, "Abstractions for network update," *ACM SIGCOMM Computer Communication Review*, vol. 42, no. 4, pp. 323–334, 2012. [Online]. Available: https://www.cs.princeton.edu/~dpw/papers/network-update-sigcomm12.pdf

[17] S.-H. Tseng, A. Tang, G. L. Choudhury, and S. Tse, "Routing stability in hybrid software-defined networks," *IEEE/ACM Transactions on Networking*, vol. 27, no. 2, pp. 790–804, Apr. 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8664455

[18] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proceedings of the 9th ACM Workshop on Hot Topics in Networks*, 2010. [Online]. Available: https://conferences.sigcomm.org/hotnets/2010/papers/a19-lantz.pdf

[19] N. Handigol, B. Heller, V. Jeyakumar, B. Lantz, and N. McKeown, "Reproducible network experiments using container-based emulation," in *Proceedings of the 8th ACM International Conference on Emerging Networking Experiments and Technologies*, 2012, pp. 253–264. [Online]. Available: https://tiny-tera.stanford.edu/~nickm/papers/mininet_hifi_conext2012.pdf